U24AI006

ASSIGNMENT -05

Consider a problem that is stated as follows. Three girls and three boys are on one side of a river, along with a boat that can hold one or two people. Find a way to get everyone to the other side without ever leaving a group of girls in one place outnumbered by the boys in that place. Formulate the problem with appropriate search space and do as per following instructions.

1. Implement and solve the problem optimally using the following search problem.
   a) Depth Limited Search (take limit=3)
   b) Iterative Deepening Search

2. Give comparison of both algorithms with the number of explored states and state approximate time complexities.

```
from collections import deque

TOTAL_G = 3
TOTAL_B = 3

MOVES = [
    (1,0),(2,0),
    (0,1),(0,2),
    (1,1)
]

def valid_state(g_left, b_left):
    g_right = TOTAL_G - g_left
    b_right = TOTAL_B - b_left

    if g_left < 0 or b_left < 0 or g_left > TOTAL_G or b_left > TOTAL_B:
        return False

    if g_left > 0 and b_left > g_left:
        return False

    if g_right > 0 and b_right > g_right:
        return False

    return True

def successors(state):
    g_left, b_left, boat = state
    next_states = []

    for g, b in MOVES:
        if boat == 0:
```

```
            new = (g_left - g, b_left - b, 1)
        else:
            new = (g_left + g, b_left + b, 0)

        if valid_state(new[0], new[1]):
            next_states.append(new)

    return next_states

def dls(state, goal, limit, path, visited, explored_count):
    explored_count[0] += 1

    if state == goal:
        return path

    if limit == 0:
        return "cutoff"

    cutoff_occured = False

    for child in successors(state):
        if child not in visited:
            visited.add(child)
            result = dls(child, goal, limit-1,
                         path+[child], visited, explored_count)

            if result == "cutoff":
                cutoff_occured = True
            elif result != None:
                return result

            visited.remove(child)

    return "cutoff" if cutoff_occured else None

def run_dls(limit):
    start = (3,3,0)
    goal = (0,0,1)
    explored = [0]

    result = dls(start, goal, limit,
                 [start], set([start]), explored)

    return result, explored[0]
```

```
result, explored = run_dls(3)
print("Result:", result)
print("Explored States:", explored)
```

OUTPUT:

```
Result: cutoff
Explored States: 10
```

CODE (2):

```
TOTAL_G = 3
TOTAL_B = 3

MOVES = [
    (1,0),(2,0),
    (0,1),(0,2),
    (1,1)
]

def valid_state(g_left, b_left):
    g_right = TOTAL_G - g_left
    b_right = TOTAL_B - b_left

    if g_left < 0 or b_left < 0 or g_left > TOTAL_G or b_left > TOTAL_B:
        return False

    if g_left > 0 and b_left > g_left:
        return False

    if g_right > 0 and b_right > g_right:
        return False

    return True


def successors(state):
    g_left, b_left, boat = state
    next_states = []

    for g, b in MOVES:
        if boat == 0:
            new = (g_left - g, b_left - b, 1)
        else:
```

```
            new = (g_left + g, b_left + b, 0)

        if valid_state(new[0], new[1]):
            next_states.append(new)

    return next_states


def dls(state, goal, limit, path, visited, explored_count):

    explored_count[0] += 1

    if state == goal:
        return path

    if limit == 0:
        return "cutoff"

    cutoff_occured = False

    for child in successors(state):
        if child not in visited:
            visited.add(child)

            result = dls(child, goal, limit-1,
                         path+[child], visited, explored_count)

            if result == "cutoff":
                cutoff_occured = True
            elif result is not None:
                return result

            visited.remove(child)

    return "cutoff" if cutoff_occured else None


def iterative_deepening():

    start = (3,3,0)
    goal = (0,0,1)

    depth = 0
    total_explored = 0
```

```
    while True:

        explored = [0]

        result = dls(start, goal, depth,
                     [start], set([start]), explored)

        total_explored += explored[0]

        if result != "cutoff" and result is not None:
            return result, total_explored, depth

        depth += 1


# RUN IDS
path, explored, depth = iterative_deepening()

print("Solution Path:")
for step in path:
    print(step)

print("Total Explored States:", explored)
print("Solution Depth:", depth)
```

OUTPUT:

```
Solution Path:
(3, 3, 0)
(3, 1, 1)
(3, 2, 0)
(3, 0, 1)
(3, 1, 0)
(1, 1, 1)
(2, 2, 0)
(0, 2, 1)
(0, 3, 0)
(0, 1, 1)
(1, 1, 0)
(0, 0, 1)
Total Explored States: 163
Solution Depth: 11
```